

Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

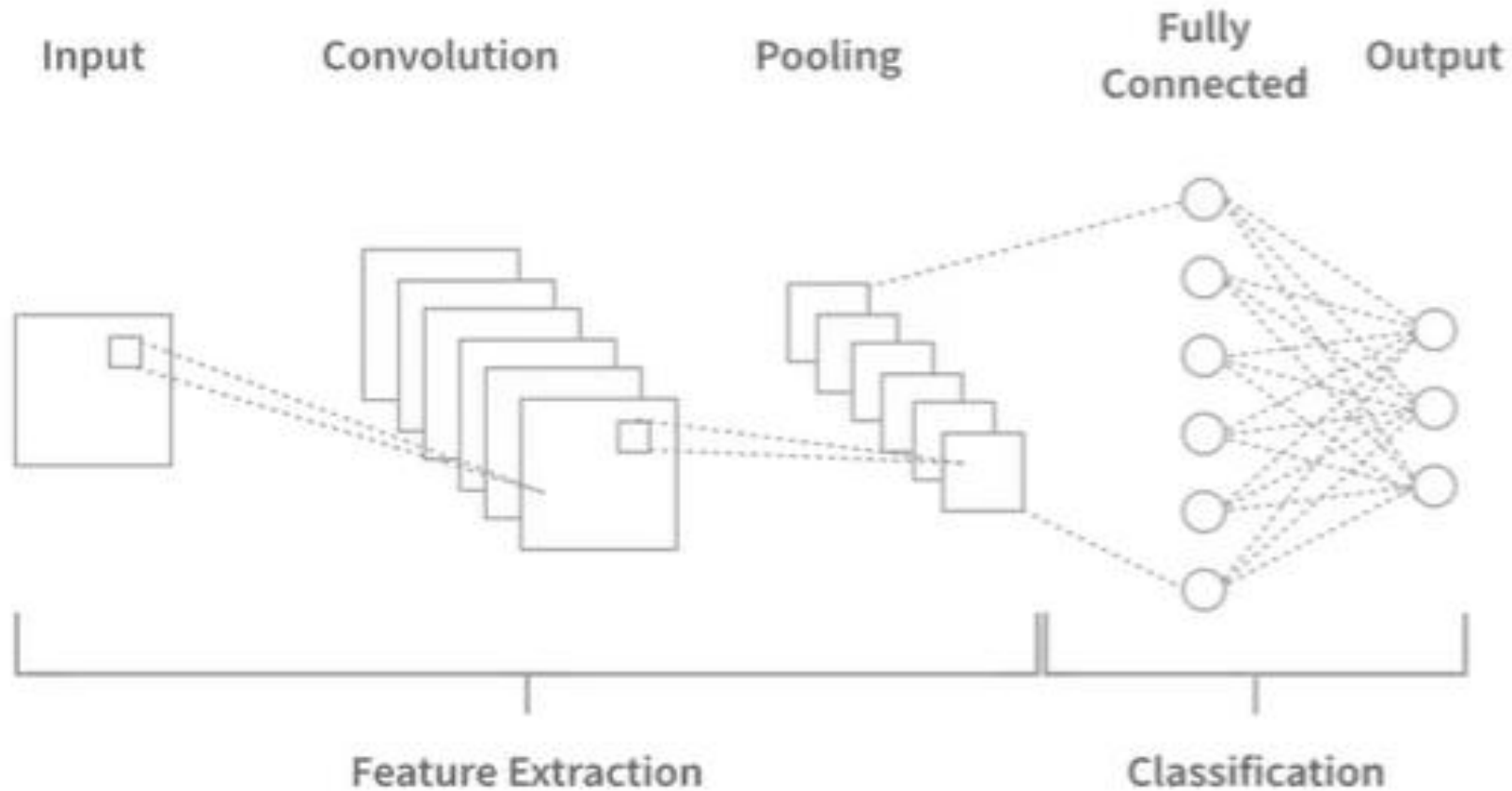
Inha University.

Email: minhaz.ahmed@gmail.com

Contents

- ▶ Neural Network Architecture
- ▶ Convolutional Neural Network
- ▶ Fully connected layer
- ▶ pooling layer
- ▶ Apply on MNIST dataset

CNN Components



A type of artificial neural network that is used to process and recognize images. CNNs are effective at finding patterns in images to recognize objects, classes, and categories.

Input layer



Where raw data, like an image,
enters the network

[illegible][illegible]

Convolutional Neural Network

- kernel(filter) and convolution

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

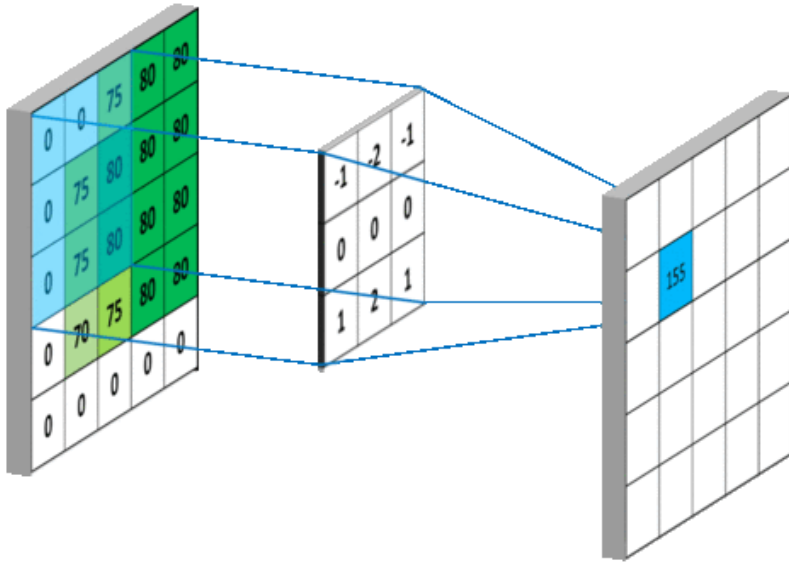
1	0	1
0	1	0
1	0	1

ex>

$$\begin{aligned} &1*1 + 1*0 + 1*1 \\ &+ 0*0 + 1*1 + 1*0 \\ &+ 0*1 + 0*0 + 1*1 \\ &= 4 \end{aligned}$$

It is also called kernel or 'filter'. Output layer is called 'feature'
This is convolution kernel. As shown in this figure, the kernel moves, and calculate convolution.

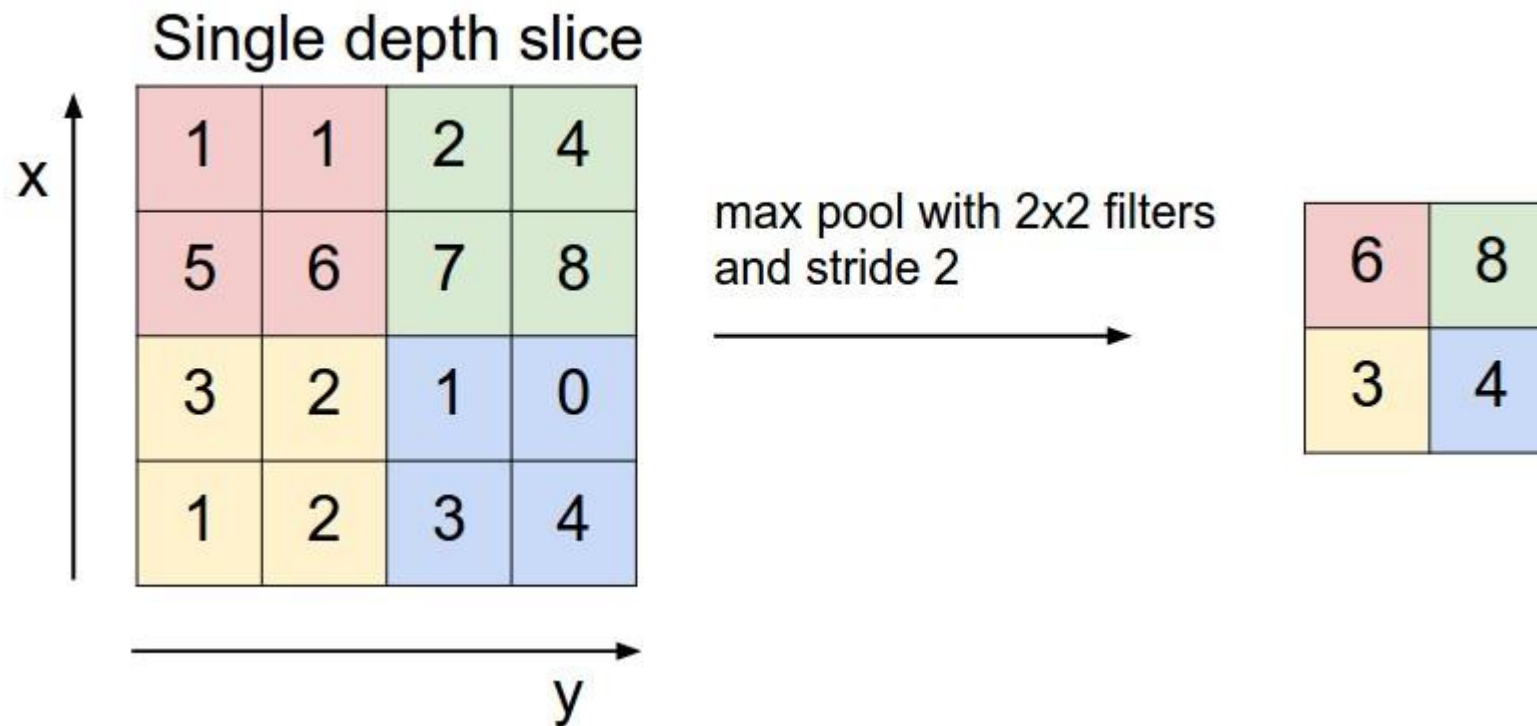
CNN layer



Convolutional Layers: Extract features from the images using convolutional filters.

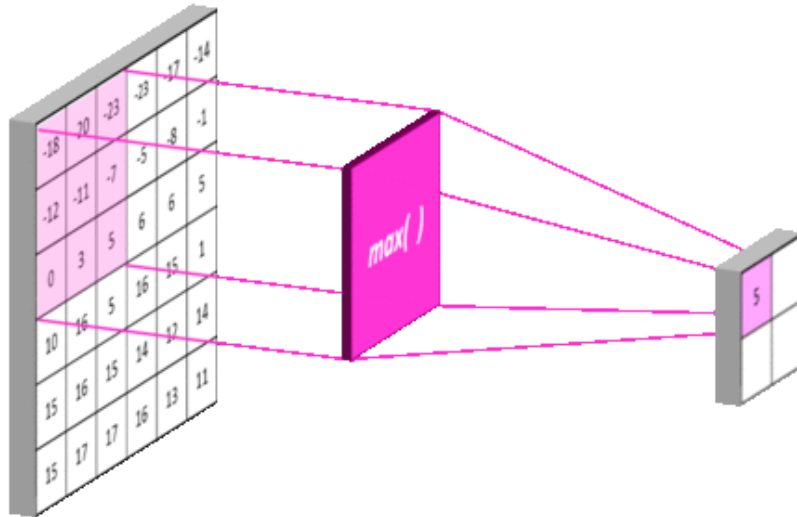
Convolutional Neural Network

- pooling



Pooling is a method of extracting elements.
This example use max pooling. Max pooling extract max value in kernel size.

Pooling layer



Max Pooling Layers: Reduce the spatial dimensions of the feature maps.

Flattening

11	119	119	251
393	119	119	260
393	190	190	260
-109	190	190	152

=

Flatten Layer: Flattens the 2D feature maps into a 1D array.

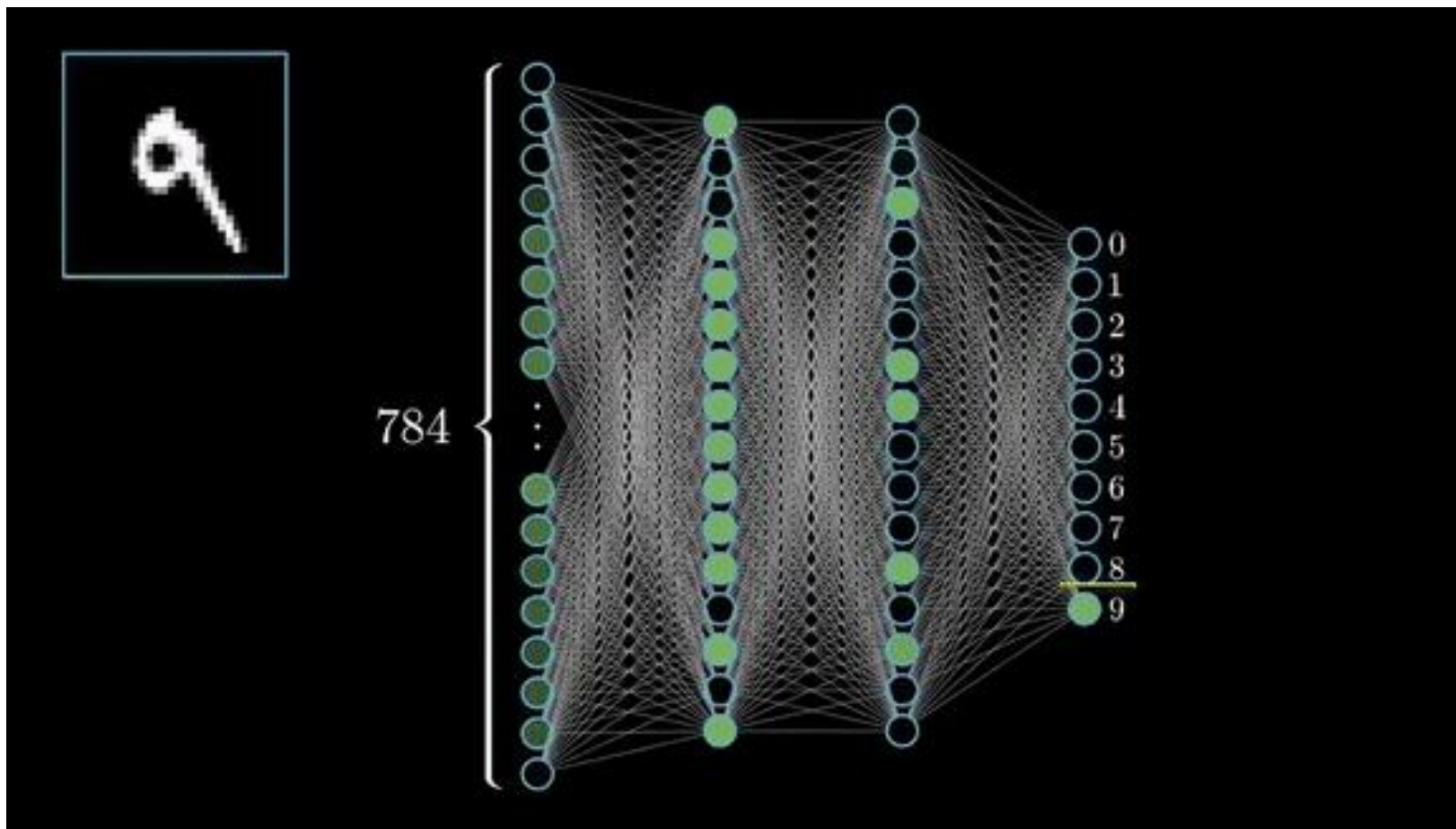
MNIST Dataset

The MNIST database contains 60,000 training images and 10,000 testing images.

the black and white images from NIST were normalized to fit into a 28x28 pixel bounding box



Classification



Practice code CNN

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Preprocess the data
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

Practice code CNN

```
# Create the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=10,
          batch_size=32, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test)
print('Test accuracy:', test_acc)
```

Why CNN

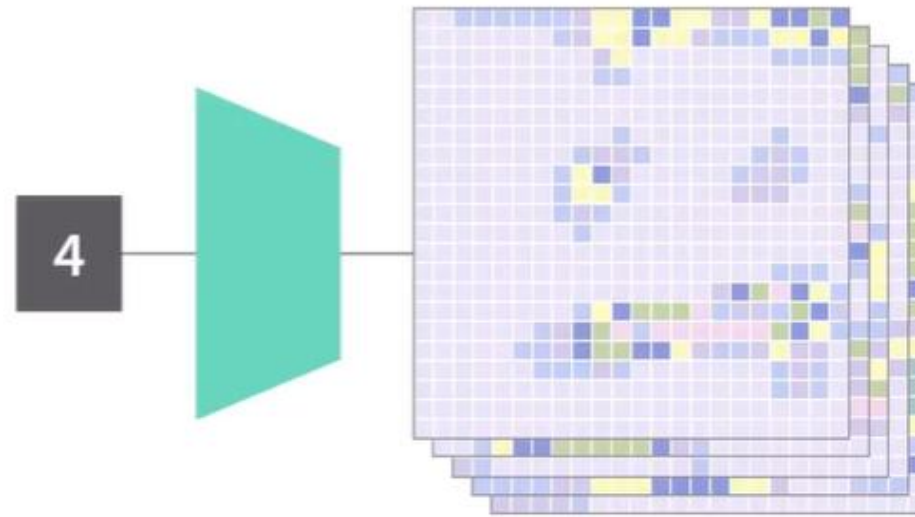
- Vanilla neural networks ignore spatial relationships in input data
- CNNs leverage spatial structure for enhanced performance
- CNNs use filters to capture local patterns and features
- CNNs excel at image classification tasks by learning spatial hierarchies
- CNNs outperform vanilla neural networks in image analysis and recognition

Parameter efficiency reduces overfitting

- CNNs exhibit parameter efficiency compared to fully connected networks
- Fully connected networks have many parameters due to connections between all neurons
- High parameter count increases computational cost and overfitting risk
- CNNs reduce overfitting by capturing local patterns and avoiding noise
- CNNs achieve efficient parameter usage through shared weights and pooling

Translation invariance

- Object position and orientation vary in image classification tasks
- Vanilla networks lack translation invariance
- Translation invariance refers to recognizing objects regardless of their position or orientation
- Vanilla networks struggle with varied object positions or orientations
- Lack of translation invariance hampers image classification performance



Translation invariance

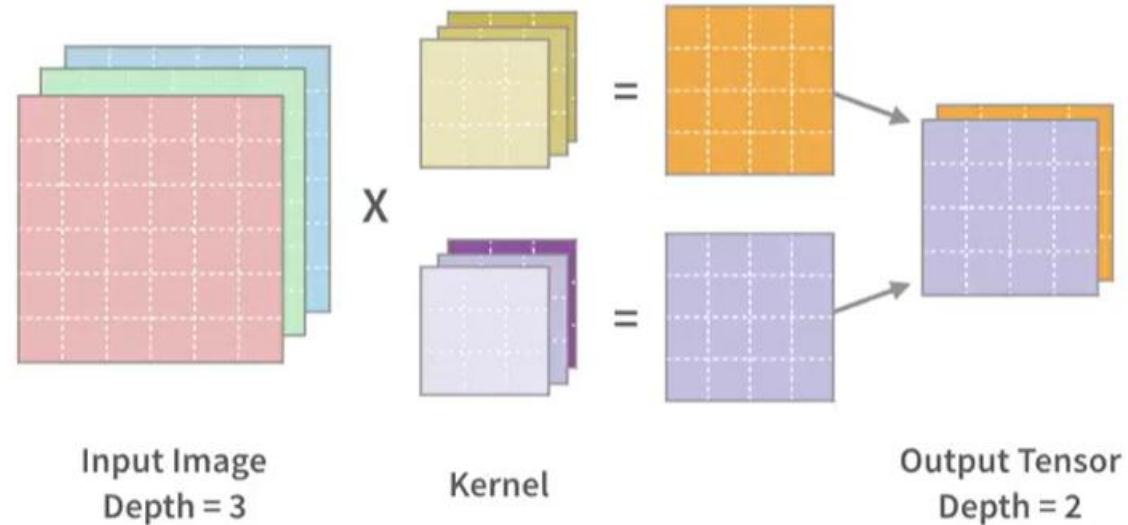
- CNNs overcome the issue through convolution and pooling layers
- Convolution and pooling enable recognition of spatial features regardless of location
- Translation invariance is crucial for robust image classification
- CNNs generalize by recognizing objects in different positions and orientations

CNN advantage

- CNNs capture and learn spatial information
- CNNs excel in recognizing complex visual patterns
- Spatial processing enables analysis of intricate visual information
- CNNs are highly effective in image classification and object recognition
- CNNs learn spatial hierarchies for tackling complex visual patterns

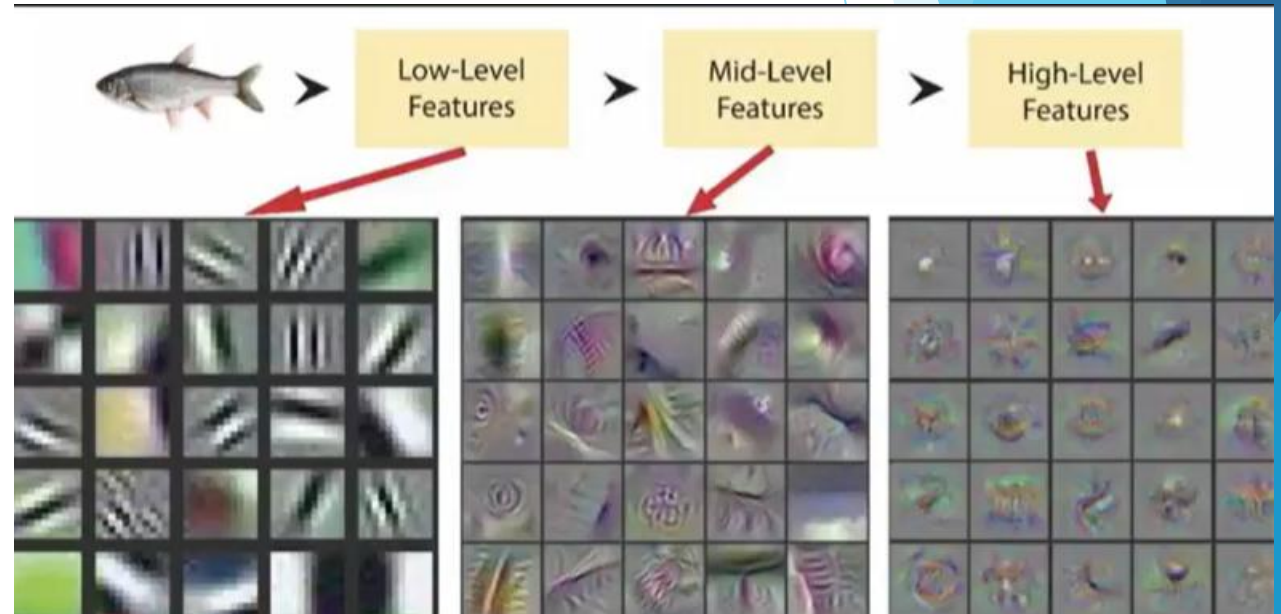
Filters and Kernels in CNN

- Filters are small matrices with trainable weights
- Filters detect specific features in the input data
- Kernels are channels within a filter
- Depth of filter matches the depth of input data



Filters and Kernels in CNN

- Filters learn complex and higher-level features during training
- Multiple filters capture various features simultaneously
- 3x3 convolutions efficiently capture local features
- They strike a balance between receptive field size and computational complexity

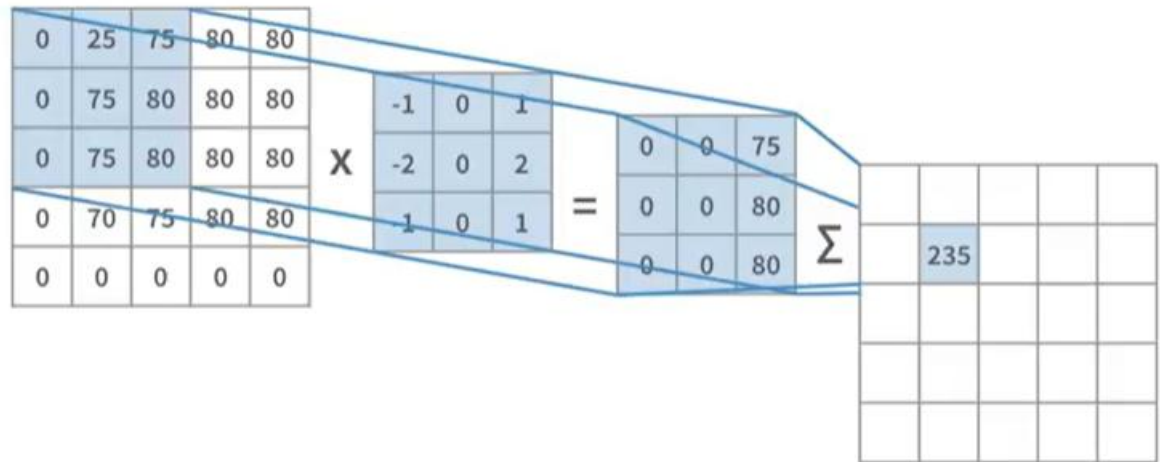


Receptive field and efficiency of 3x3 convolution

- Receptive field refers to the input region contributing to a feature in the output
- A 3x3 kernel captures immediate neighboring information with fewer parameters
- Stacking two 3x3 convolution layers is as effective as a single 5x5 layer
- Two 3x3 layers provide efficiency with fewer parameters and computations

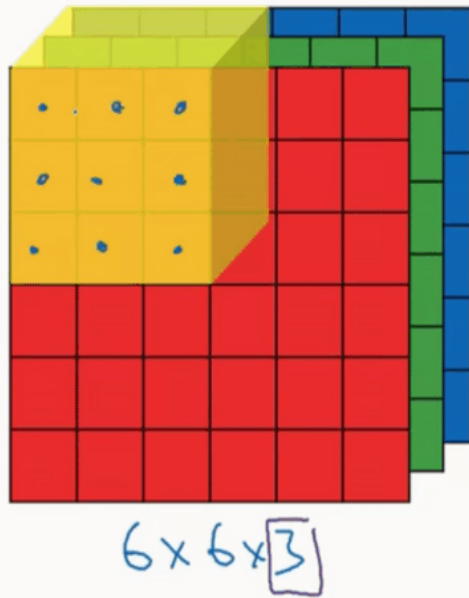
Convolutions

- Convolution in CNNs slides a filter/ kernel over input data
- The filter is applied to local regions in a sliding window manner
- Element-wise, multiplication is performed between the filter and the input
- Results are summed to produce a value in the output feature map

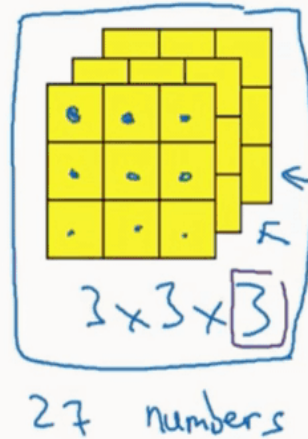


Convolution on RGB image

Convolutions on RGB image



*

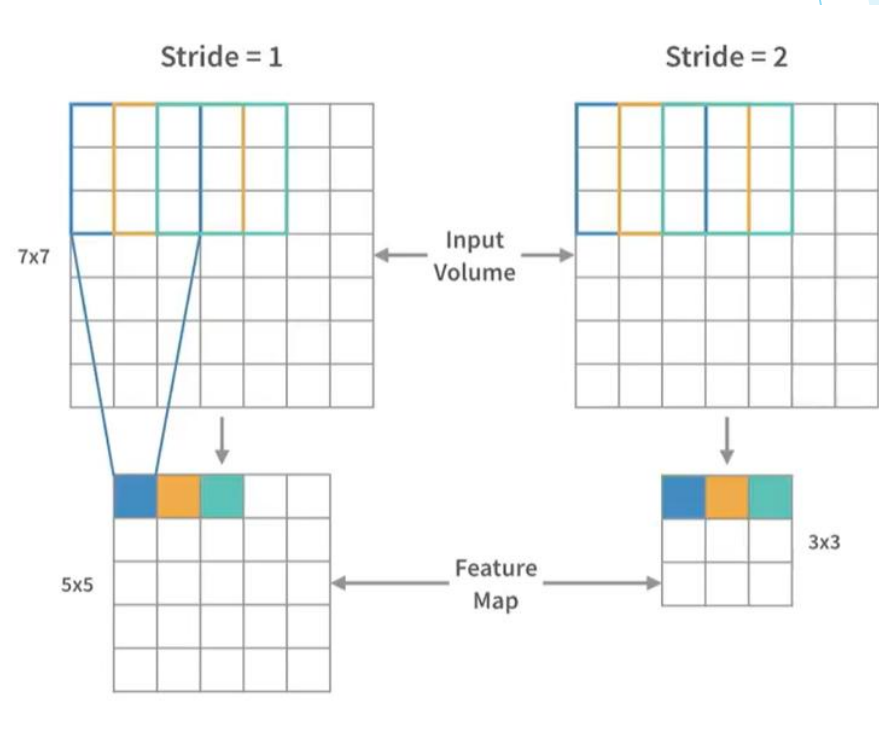


=



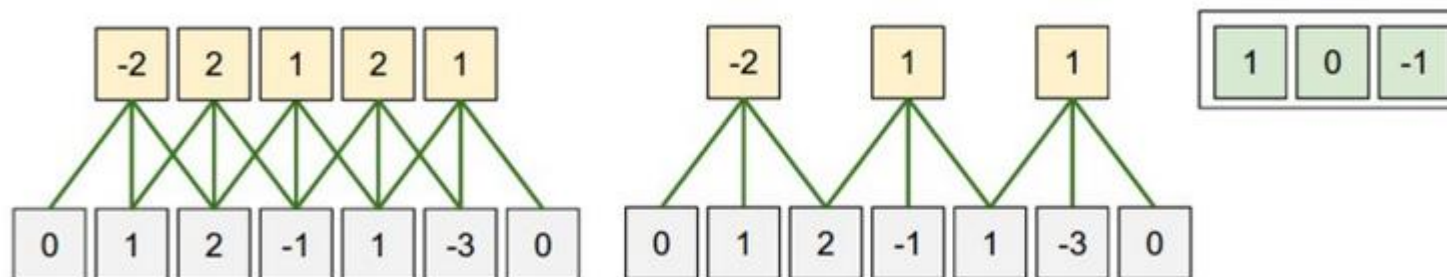
Stride in CNN

- Stride is the number of pixels the filter moves across the input data
- A stride of one results in densely computed output feature maps
- Larger strides skip pixels, creating a sparser output
- Larger stride improves computational efficiency but may sacrifice spatial information



Convolutional Neural Network

- stride



This is stride. Stride is a factor determining how much the kernel will move at a time. For example, we use this kernel. And left, using stride equal 1, right using stride equal 2.

Convolutional Neural Network

- padding

- Convolution with zero-padding of P pixels

x1	x0	x1
x0	x1	x0
x1	x0	x1

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	1	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

2	2	3	1	1
1	4	3	4	1
2	2	4	3	3
1	2	3	4	1
1	2	3	1	1

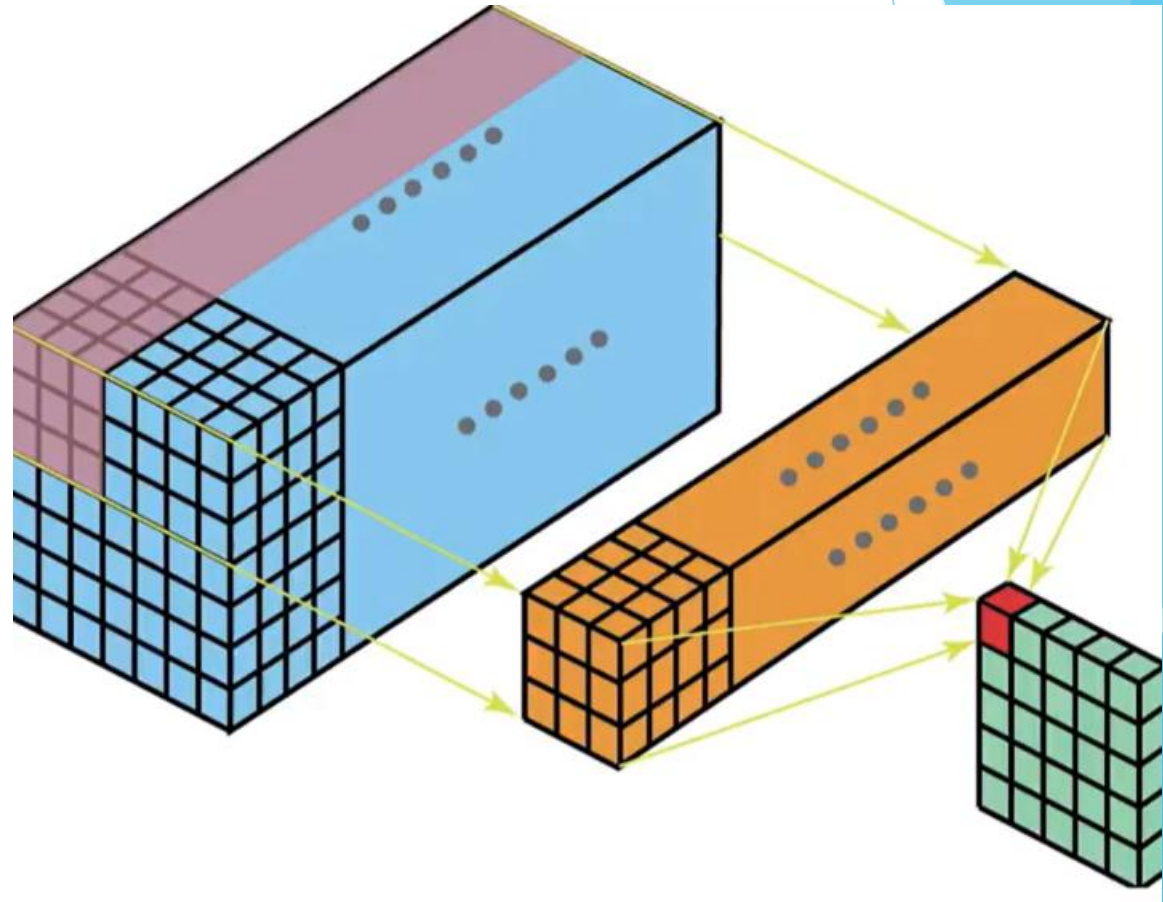
If we calculate convolution, output feature is smaller than input layer.

So we use 'padding' to stay in feature size. For example, this feature has surround by zeros. These zeroes serve as maintain the size of the feature.

Types of Convolutions

Standard 2D convolution

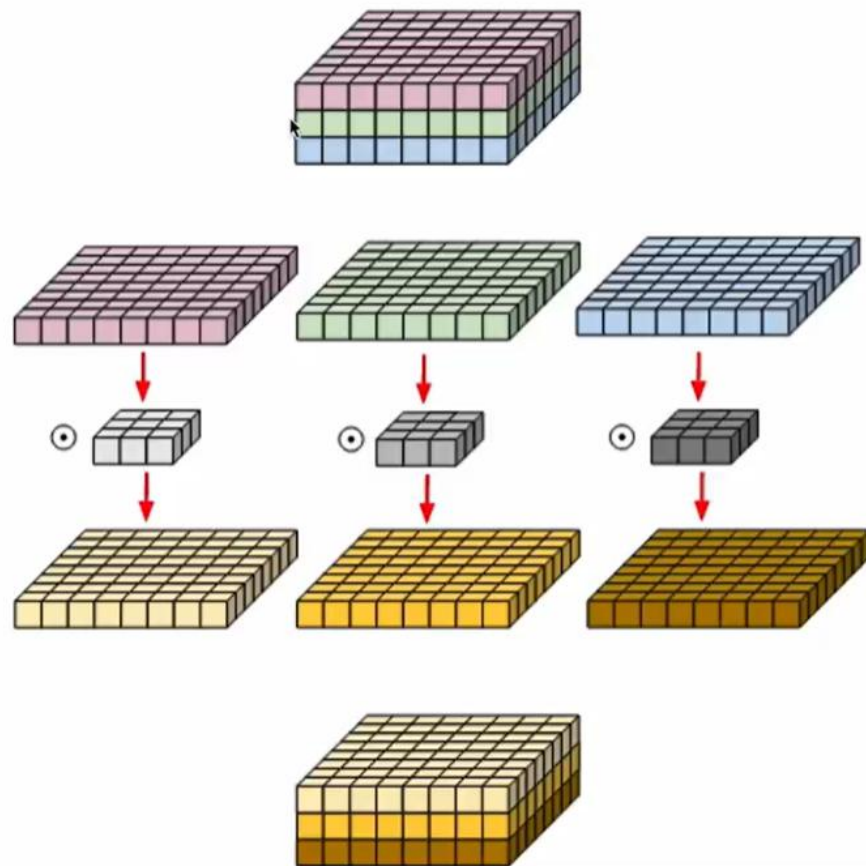
- Widely used type of convolution in CNNs
- 2D filter slides over 2D input, for example, an image
- Element-wise multiplication and summation for output
- Captures spatial features like edges, corners, and textures



Depth wise convolution

- Depthwise convolution is a computationally efficient variant
- Each input channel is convolved with its corresponding filter
- Output feature maps are stacked together for the final output
- Reduces computations and parameters, enhancing speed and memory efficiency
- Effectively captures spatial information

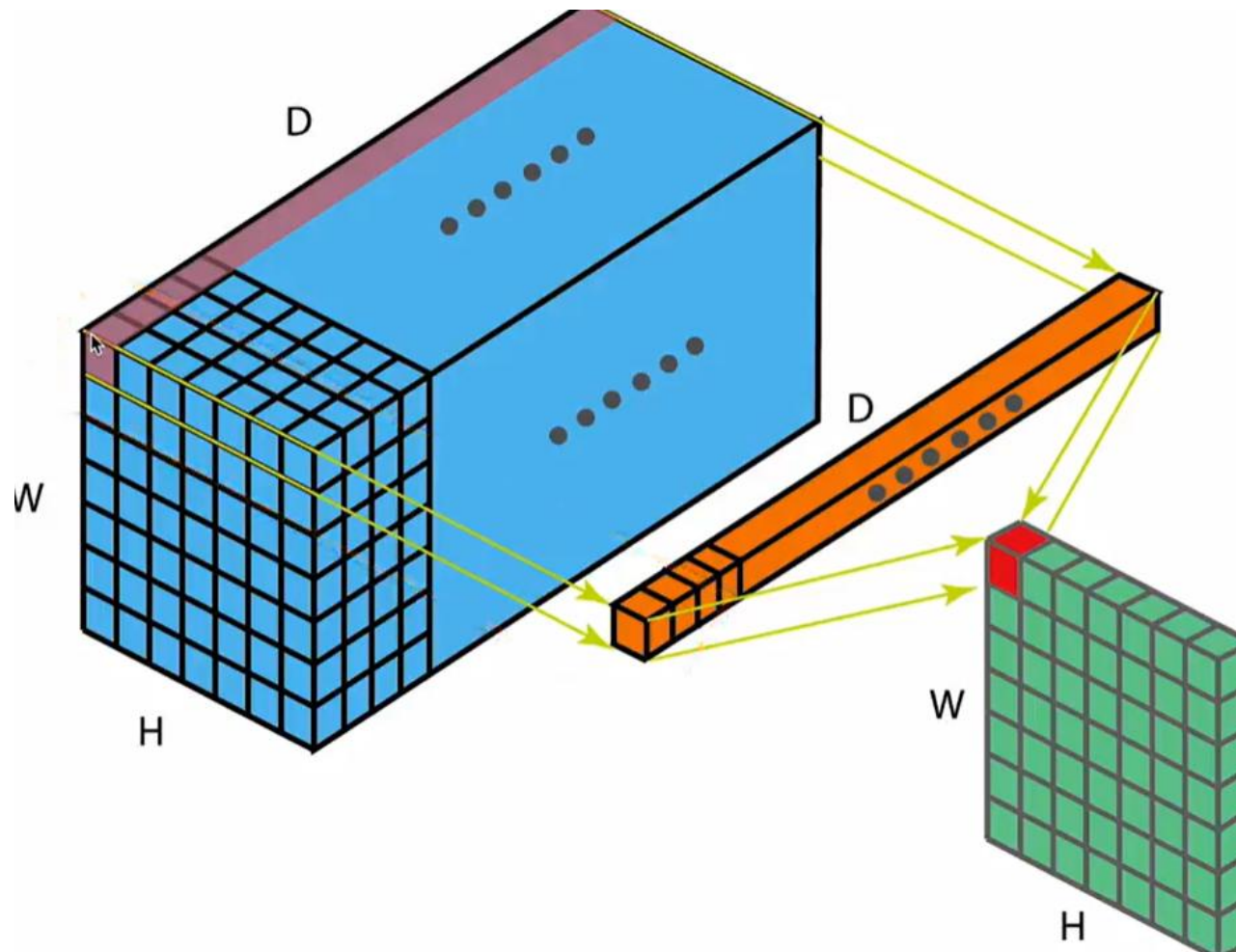
Depth wise convolution



Point wise convolution

- 1x1 convolutions change feature map depth without altering spatial dimensions
- They reduce channels or combine information
- Computationally efficient, often used with depthwise convolutions
- In complex architectures, they reduce computational complexity

Point wise convolution



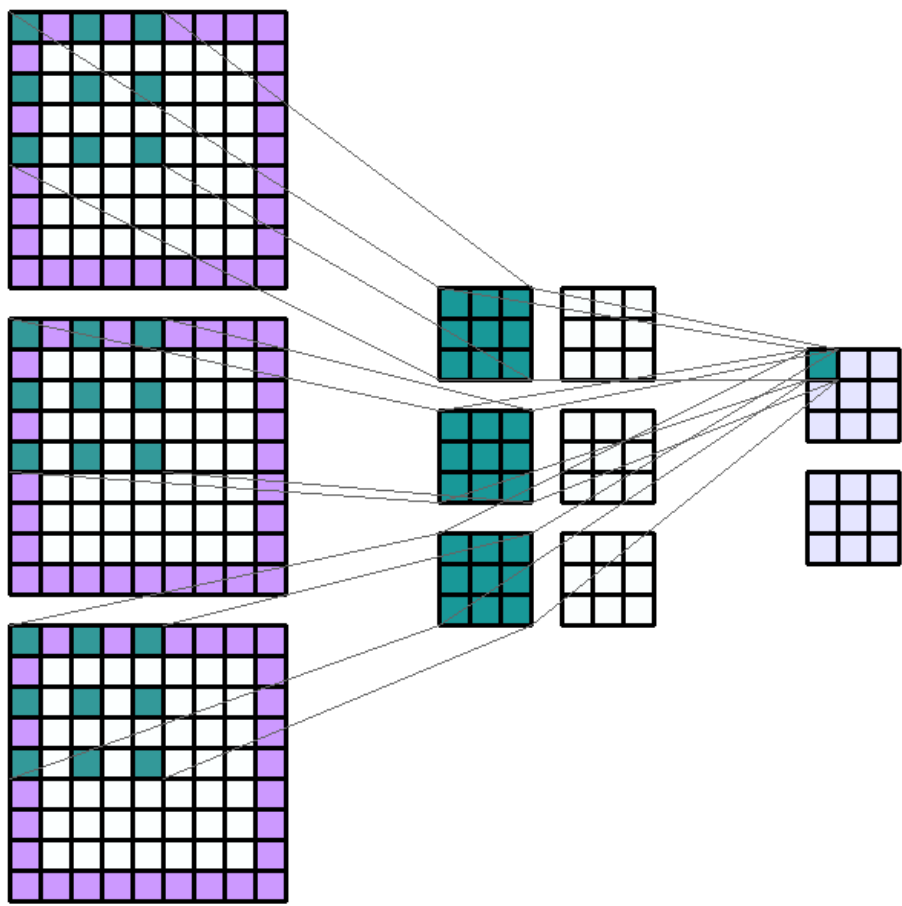
Depth wise separable convolution

- Combines depthwise and pointwise convolution
- Depthwise convolution captures spatial information
- Pointwise convolution increases depth and combines features
- Reduces computational complexity compared to standard convolution
- Popular in lightweight models like MobileNet

Dilated convolution

- Dilated convolution introduces a dilation factor
- Dilation expands receptive field without increasing size or parameters
- Useful for capturing information at multiple scales or resolutions
- Applied in tasks like image segmentation and object detection

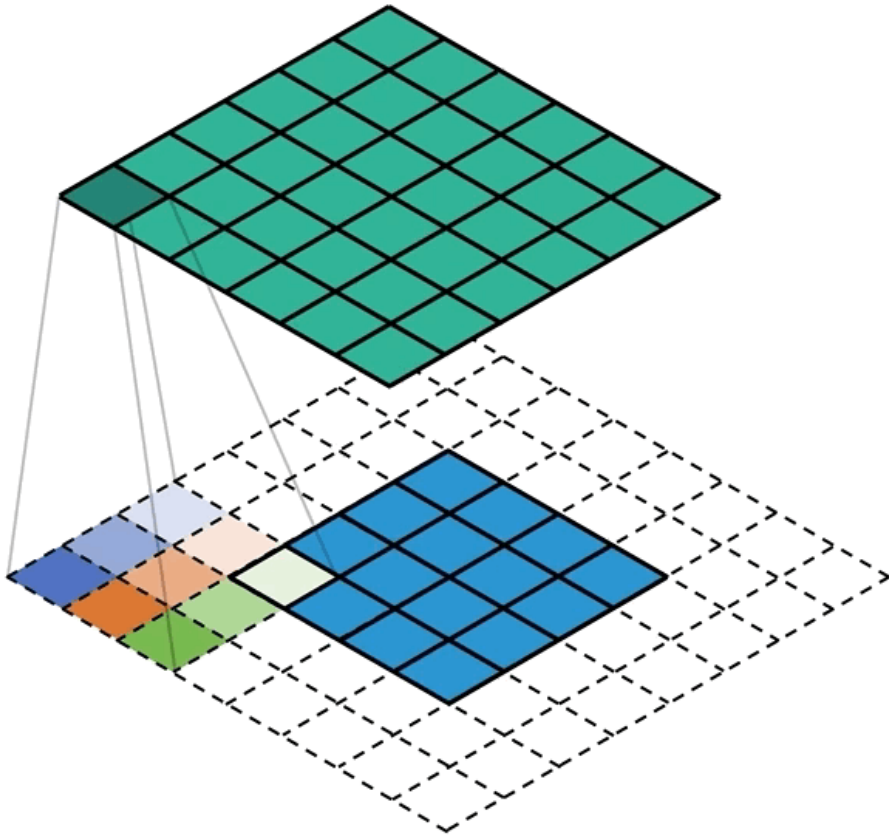
Dilated convolution



Transposed convolution

- Transposed convolution increases spatial dimensions (upsampling)
- Commonly used in tasks like image segmentation and generative models
- Goal is to produce higher-resolution output than the input
- Learnable filter applied to expand, not reduce, spatial dimensions

Transposed convolution



Types of Convolutional layer summary

- 2D convolutions are widely used for capturing spatial features
- Depthwise convolutions efficiently capture spatial information
- Pointwise convolutions manipulate depth and enhance computational efficiency
- Depthwise separable convolution combines efficiency and lightweight design
- Dilated (atrous) convolution expands the receptive field for multiscale information capture

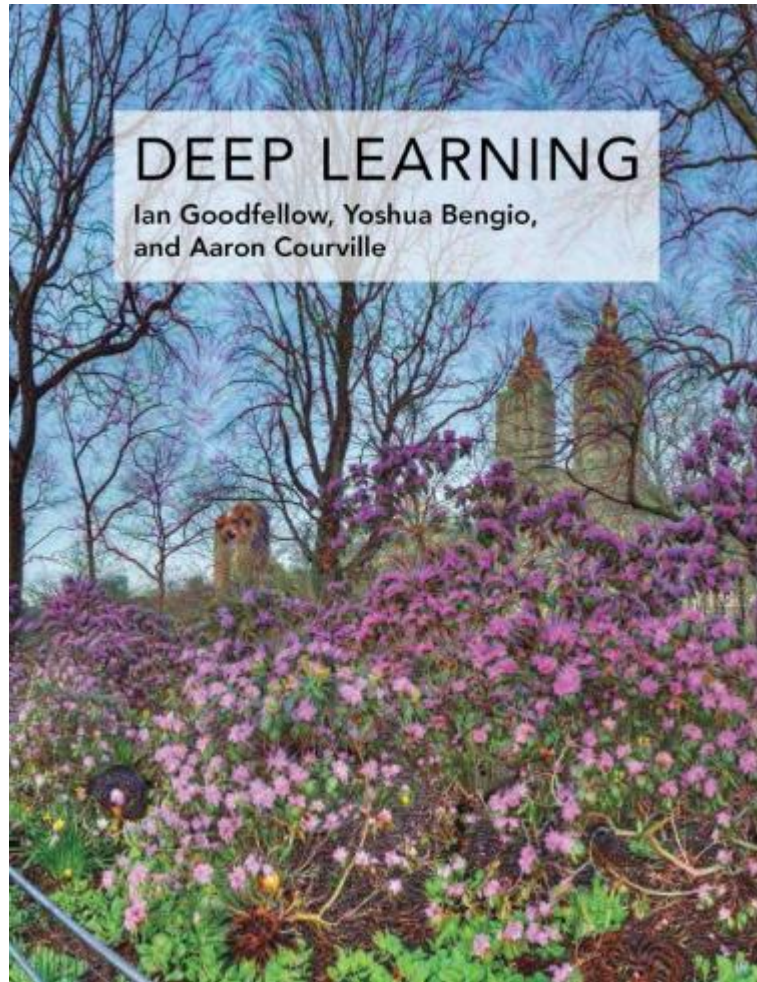
Pooling layers

- Pooling layers reduce feature map size, improving efficiency
- Pooling adds spatial invariance, enhancing generalization
- Hierarchical feature learning combines pooling and convolutional layers
- Deeper layers have wider receptive fields and learn higher-level features
- Complex representations from input data lead to improved performance

Max pooling

- Max pooling is the most commonly used pooling operation
- A small window slides over the input feature map
- Keeps the maximum value in the window
- Retains prominent features while discarding less important information
- Robustness to small translations maintains feature capture and output stability

Reference



Reference

- ▶ <https://www.pyimagesearch.com/2020/03/16/detecting-covid-19-in-x-ray-images-with-keras-tensorflow-and-deep-learning/>
- ▶ <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

Question



Thank you

